INSTITUT NATIONAL DES SCIENCES APPLIQUÉES LYON

IMAGE ANALYSIS

PIPELINE FOR FREE HANDWRITTEN TEXT RECOGNITION

Student : NGUYEN Sang

January 2026

Contents

1. Introduction **1**
2. Data Acquisition and Dataset Construction **2**
2.1. Manual handwriting and scanning 2
2.2. Image preprocessing 2
3. Segmentation **4**
3.1. Line Segmentation (Horizontal Projection) 4
3.2. Character Isolation and Component Merging 4
3.3. Data Normalization and Resizing 7
4. Feature Extraction **8**
4.1. Feature Descriptors 8
4.2. Analysis 8
5. Model training **10**
5.1. Classifier Training 10
5.2. Analysis 10
6. Recognition Errors and Potential Causes **11**
6.1. Morphological Ambiguity 11
6.2. Disconnected Components (i and j) 11
6.3. Segmentation Failure in Complex Layouts 11
6.4. Thin Strokes 11
7. Final Recognition Results and Evaluation **13**

1. Introduction

The objective of this project is to implement a complete handwritten text recognition (HTR) pipeline, covering complete stages from data acquisition, classification to evaluation. This project concentrates on classical computer vision pipeline including:

- **Data Acquisition:** Manual handwriting of the alphabet and reference texts and high-resolution scanning (at best attempt).
- **Preprocessing:** Refining raw images into clean binary data through a sequence of: Grayscale Conversion, Gaussian Blurring and Otsu's Binarization.
- **Hierarchical Segmentation:** A top-down approach to separate characters: Line Detection, Character Isolation and Component Merging.
- **Feature Extraction:** Transforming normalized 32×32 character images into high-dimensional numerical descriptors using a fusion of **Hu Moments** and **HOG (Histogram of Oriented Gradients)**.
- **Classification & Evaluation:** Label prediction using **SVM** and **Random Forest** models, followed by text reconstruction and performance analysis.

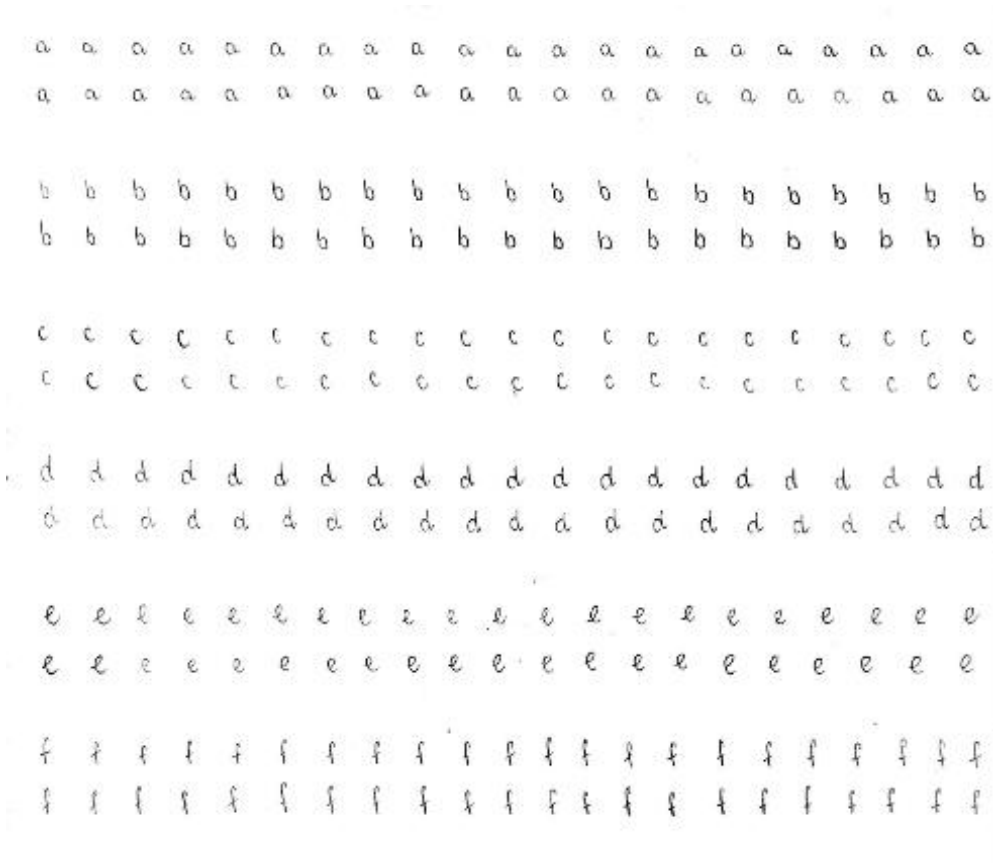
2. Data Acquisition and Dataset Construction

2.1. Manual handwriting and scanning

A reference text for testing and the complete alphabet (from **a** to **z**) were manually written on a blank sheet using a consistent handwriting style. The sheet was then scanned at high resolution to minimize noise and preserve stroke details.

The alphabet was written in the following format:

- Each letter was written on **two separate lines**.
- A **horizontal spacing of at least 0.5 cm** was maintained between consecutive characters.
- A **vertical spacing of approximately 1 cm** was used between different letters.



This layout ensures clearer separation between characters and helps segmentation and dataset construction processes.

2.2. Image preprocessing

Each scanned image goes through the preprocess state including: grayscale conversion, noise reduction (Gaussian blurring), and Otsu adaptive thresholding to to handle potential non-uniform lighting in the scanned document.

```python
def preprocess(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    blur = cv2.GaussianBlur(gray, (3,3), 0)
    _, bin_img = cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
    return bin_img
```

a) Grayscale conversion

cv2.cvtColor(img, cv2.COLOR_BGR2GRAY): converts the original color image (BGR) into grayscale.

Reason: In HTR, grayscale reduces computation and simplifies the thresholding step by reducing the number of color channels of the image.

b) Noise reduction

cv2.GaussianBlur(gray, (3,3), 0): applies a Gaussian blur with a 3x3 kernel.

Reason: Smooth out small noises (paper texture in this project) in the image that can create false contours or small blobs.

c) Otsu's adaptive thresholding

cv2.threshold(blur, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU): converts the image to binary: pixels become either black (0) or white (255).

THRESH_BINARY_INV inverts colors (text becomes white, background black).

THRESH_OTSU chooses the best threshold for separation, handling non-uniform lighting.

Reason: Helps detecting contour easier. Otsu adapts to light variations, which is common in scanned documents.

3. Segmentation

Segmentation step helps convert raw handwritten text of the alphabet into individual character images that can be classified by machine learning models. This pipeline follows a top-down approach: document to lines to separate characters.

3.1. Line Segmentation (Horizontal Projection)

As the characters are written in multiple lines, using Horizontal Projection Profiles is necessary:

```
def split_lines_dynamically(bin_img, min_height=20):
    proj = np.sum(bin_img, axis=1)
    lines = []
    in_line = False
    start = 0
    threshold = 10

    for y, val in enumerate(proj):
        if val > threshold and not in_line:
            in_line = True
            start = y
        elif val <= threshold and in_line:
            in_line = False
            end = y
            if (end - start) > min_height:
                lines.append(bin_img[start:end, :])
    if in_line:
        lines.append(bin_img[start:, :])
    return lines
```

a) Horizontal projection

np.sum(bin_img, axis=1): Horizontal projection

Due to THRESH_BINARY_INV, rows with text have more white pixels. Therefore, adding up pixel values along each row helps determine which row belongs to a line.

b) Line detection logic

If a row has enough white pixels (above threshold), it's part of a line.

If a row is mostly black (below threshold), the line ended.

Only lines taller than min_height are kept.

Low-value areas (rows with sum values below threshold) are used as boundaries to split the image into separate lines. Instead of relying on fixed-size cropping which is susceptible to error, the algorithm detects line boundaries dynamically to solve the issues of: varying line heights, uneven spacing, and slight handwriting slant.

3.2. Character Isolation and Component Merging

After line segmentation, the pipeline becomes:

- Stack two lines together
- Find connected components (contours)

- Convert contours to bounding boxes
- Merge boxes that belong to the same *character*
- Crop, resize, and pad each character

```
def extract_chars_from_two_lines(line1, line2, letter=None, min_area=20):
    combined = np.vstack([line1, line2])
    contours, _ = cv2.findContours(combined, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    # 1. Get initial bounding boxes
    rects = []
    for cnt in contours:
        x, y, w, h = cv2.boundingRect(cnt)
        if w*h < min_area:
            continue
        rects.append([x, y, w, h])

    if not rects:
        return []
```

a) Stack two lines together

Vertically stacks line1 on top of line2 to get a shape of (height1 + height2, width). This allows dot and stem in case of **"i"** and **"j"** to become closer and able to merge later.

b) Find contours

Then the process finds connected groups of white pixels (contours):

_RETR_EXTERNAL: only keeps outermost blobs and avoids nested contours (holes inside letters like **"o"**, **"a"**).

_CHAIN_APPROX_SIMPLE: compresses contour points to make the process faster, consumes less memory, while still remaining the same bounding box.

c) Convert contours to bounding boxes

Then while removing tiny blobs of noises such as dust, stray pixels or paper textures (if the area of the blob is smaller than the min_area), save bounding boxes for further matching

d) Merge boxes that belong to the same character

```python
# 2. Merge overlapping components (for i and j with dots)
rects = sorted(rects, key=lambda r: r[0])
merged_rects = []
used = set()

for i in range(len(rects)):
    if i in used: continue
    curr_x, curr_y, curr_w, curr_h = rects[i]

    for j in range(len(rects)):
        if i == j or j in used: continue
        next_x, next_y, next_w, next_h = rects[j]

        # If they overlap horizontally, they are part of the same letter (like the dots of i and j)
        x_overlap = not (next_x > curr_x + curr_w + 2 or curr_x > next_x + next_w + 2)

        if x_overlap:
            # Expand current box to encompass the new piece
            new_x = min(curr_x, next_x)
            new_y = min(curr_y, next_y)
            new_w = max(curr_x + curr_w, next_x + next_w) - new_x
            new_h = max(curr_y + curr_h, next_y + next_h) - new_y

            curr_x, curr_y, curr_w, curr_h = new_x, new_y, new_w, new_h
            used.add(j)

    merged_rects.append([curr_x, curr_y, curr_w, curr_h])
    used.add(i)
```

Some characters, such as “i” and “j”, are composed of multiple disconnected parts (a stem and a dot). However, there’s a solution to this problem. Letters are separated horizontally so the dot and the stem have the same x-range but differ in y. Therefore, if two bounding boxes overlap horizontally, they belong to the same character. Meaning if $\max(x_1, x_2) \leq \min(x_1 + w_1, x_2 + w_2)$.

e) Crop, resize and pad each character

```python
# 3. Final extraction and padding
merged_rects = sorted(merged_rects, key=lambda r: r[0])
final_chars = []
for x, y, w, h in merged_rects:
    char_img = combined[y:y+h, x:x+w]
    padded = resize_and_pad(char_img)
    if padded is not None:
        final_chars.append(padded)

return final_chars
```

After sorting left to right to ensure correct reading order, the pipeline then extracts a tight character image, keeps aspect ratio and centers character. The output is an image of the character of size 32x32.

3.3. Data Normalization and Resizing

Before feeding the dataset into the SVM and Random Forest classifiers, all images must share the same format.Thanks to resizing step in ***3.2. e)***, characters are resized proportionally and scaled and centered on a **32×32 black canvas**. This normalization step helps improve classifier stability and performance.

4. Feature Extraction

Feature Extraction converts raw pixel images into meaningful numerical features that machine learning models can understand. We tried to find features that can capture both the overall shape and the small stroke details of the letters. To find the best method, we tested three different feature extraction methods, as well as the combination of the best ones.

4.1. Feature Descriptors

a) Geometric Features (3 features)

This set includes 3 features: Aspect Ratio, Extent, and Solidity that describe the size, shape, and pixel density of a character inside its bounding box. They are fast to compute but contain limited information.

b) Hu Moments (7 features)

Hu Moments are global shape descriptors that are invariant to translation, rotation, and scale. They summarize the overall structure of a character. To avoid numerical instability caused by very large or very small values, we need to apply a logarithmic normalization:

$$f_i = -\operatorname{sign}(h_i)\log_{10}(|h_i| + 10^{-10})$$

This transformation keeps the values in a stable and comparable range.

c) Histogram of Oriented Gradients (HOG) (324 features)

HOG focuses on local edge information by computing gradient orientations within small 8×8-pixel cells. This method captures stroke directions, curves, and corners, which are crucial for distinguishing handwritten letters with similar global shapes.

d) Combined Feature Fusion (331 features)

This approach concatenates HOG and Hu Moments into a single feature vector. The goal is to combine global shape information with detailed local stroke information, producing a representation of each character that contains more information.

4.2. Analysis

The experimental results are summarized below:

Method	Feature_Count	Avg_Time_ms	SVM_Accuracy	RF_Accuracy
Geometric	3	0.0055	0.4858	0.6073
Hu	7	0.0046	0.7484	0.8694
HOG	324	0.0717	1.0	0.9976
Combined	331	0.0819	1.0	0.9976

For HTR, we want to focus on the accuracy of the predictions rather than computation time. In this context, computational cost is a secondary concern, especially since feature extraction is performed offline during training or once per image during inference.

Although both the HOG-only and the combined feature approaches achieved the same accuracy on the classification results of the alphabet dataset, we choose to use the combined representation mainly because it provides a more stable description of handwritten characters. By integrating global shape information (Hu Moments) with local stroke-level details (HOG), the combined features offer greater robustness to variations in writing style, stroke thickness, and minor segmentation errors.

5. Model training

5.1. Classifier Training

Two models were trained on the alphabet dataset:

1. **Support Vector Machine (SVM):** Implemented with an **RBF kernel** and C=10. A StandardScaler was applied to normalize the HOG and Hu vectors, as SVMs are sensitive to feature scales.
2. **Random Forest (RF):** Configured with 300 trees. This ensemble method was used to evaluate non-linear decision boundaries without the strict requirement for feature scaling.

5.2. Analysis

In comparison from the same performance table above and a small test below with provided groundtruth: “thequickbrownfoxjumpsoverthefence”, SVM performed better for high-dimensional data (HOG).

```
SVM: thequickbrownfoxjumpsoverthefence
RF : theqvickbrownfoxjvmpsoverthefence
```

It achieved a greater accuracy for this dataset and performed in practice generally better in this context. Random Forest was more effective when using lower-dimensional, "weak" features like Geometric and Hu Moments, outperforming SVM in those specific categories. However, as the chosen feature extraction method provides a high-dimensional data, for the final pipeline, **SVM with Combined Features** was selected as the primary classifier due to its superior precision.

6. Recognition Errors and Potential Causes

While the system achieved high performance on the isolated alphabet dataset, testing on real continuous text (as seen in full_text.jpg) revealed several edge cases.

6.1. Morphological Ambiguity

One of the most frequent errors is the confusion between characters with high structural similarity.

- **Observation:** The classifier often misidentifies a 'u' as a 'v'.
- **Possible cause:** In fast handwriting, the rounded base of the 'u' becomes sharper, making its **HOG (Histogram of Oriented Gradients)** signature nearly identical to a 'v'. Since the model relies on local edge orientations, the lack of a clear horizontal gradient at the bottom of the 'u' leads to misclassification.

6.2. Disconnected Components (i and j)

Despite the implementation of a merging method in our segmentation pipeline, 'i' and 'j' remain problematic.

- **Observation:** The "dot" is sometimes treated as a separate character (like a noise speck) or the stem is recognized as an 'l'.
- **Possible Cause:** If the vertical gap between the stem and the dot exceeds the heuristic threshold, the extract_chars_from_two_lines function fails to merge them. Conversely, if the writing is too slanted, the dot may align horizontally with the *next* character, causing an incorrect merge.

6.3. Segmentation Failure in Complex Layouts

There are contributing factors in real recognition text such as noises and uncovered layout structure in the segmentation process.

- **Noise and stray marks:** Scanned dots, or textures from the paper grains during camera capture are often captured as "contours." Even with a min_area filter, large noise particles can be mistaken for small letters.
- **Touching characters:** In the phrase *"the quick brown fox"*, letters often touch (e.g., 'th' or 'ck').
  - **Result:** The **Horizontal/Vertical Projection** fails to find a "zero-sum" gap to split the letters.
  - **Impact:** The feature extractor receives a "double-letter" image. Since the model was only trained on single characters, it produces a "garbage" prediction.

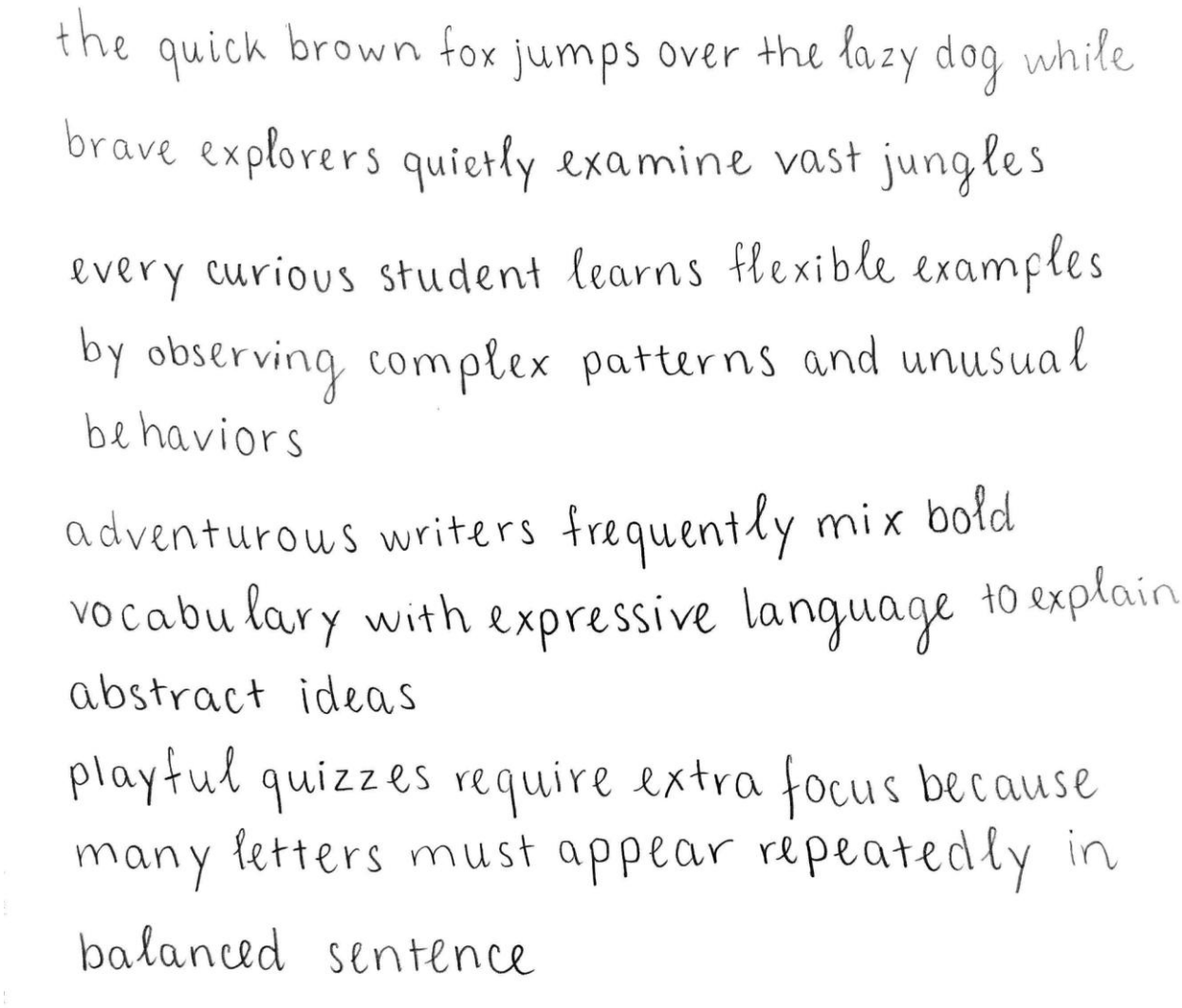
6.4. Thin Strokes

- **Observation:** Some characters appear broken after binarization.

- **Possible cause: Otsu's Thresholding** assumes a bimodal distribution of pixel intensities. If the pen pressure was light (thin strokes), the global threshold might categorize those light gray pixels as background (white), and as a result, it effectively "cuts" the letter into pieces and destroys its HOG signature.

7. Final Recognition Results and Evaluation

The pipeline was executed on the continuous text sample using the **Combined (HOG + Hu)** feature set.



The resulting raw strings from both classifiers are presented below:

- **SVM Output:**tkequickbrownfoxjumpsovertvbzydogmilebraveexplorersqoietlyexannineva stjvngleseverywriovsstudentlearnsflexjbeexamplesbyobservjjfgconnplexpatvrnsand unusualblhaviorszzztizsjzezjpjxgxjxadmrabstractideasitzijzzezjjjfxnxxsibalanadsen xnu
- **Random Forest (RF) Output:**tkequickbrownfoxjumpsoverdubzydogmilebraveexplokersqagetlyexannjne vastjvngleseveryariovsstudentlearnsflexgbkexamplesbyobslrwgjqgconnpelxpatarnsa

ndunasualblhaviorsziijjcsiiijipjzqeizwdmrabstractideasijfijciezjjjfxoaasibalanwdsen knu

Both models could recognize high-frequency words like *"quick"*, *"brown"*, and *"fox"*. However, the **SVM** model demonstrated a higher degree of structural consistency. For example, in the phrase *"jumps over the"*, the SVM outputted jumpsovertv, whereas RF produced jumpsoverdub, showing that SVM is slightly better in handling high-dimensional gradient features.

The clusters of characters such as zzztizsjzez at the end of lines are wrong predictions. These correspond to noise in the original scan that were not successfully filtered by the min_area heuristic during the segmentation stage.

The string qoietlyexannine (intended as *"quietly examine"*) illustrates the "i" vs "u" and "m" vs "nn" ambiguity discussed in Section 6. Because the system recognizes characters individually, it lacks the linguistic context to correct exannine back to examine.